PARTICULARS OF THE EXPERIMENTS PERFORMED CONTENTS

SL no.	Date	Experiments	Page no.
1		**Statistical, Pandas, NumPy Functions**	
2		**Basic operations and Filtering data**	
3		**Reading and Displaying files in tkinter**	
4		**Data Visualization using Scatter plot, Bar chart, Line graph, and Pie chart**	
5		**Confusion matrix**	
6		**Data Preprocessing**	
7		**KNN classification**	
8		**Linear Regression**	
9		**K – Means Clustering**	
10		**Naive Bayes**	

PRG1: Write a Python program to Illustrate statistical functions

a) Using Statistics Library
b) Using Pandas
c) Using Numpy

a) Using Statistics Library

```
import statistics
var = [20,30,40,50,30]
a = statistics.mean(var)
b = statistics.median(var)
c = statistics.mode(var)
d = statistics.stdev(var)

print("Mean=",a)
print("Median=",b)
print("Mode=",c)
print("STDEV",d)
```

```
Mean= 34
Median= 30
Mode= 30
STDEV 11.40175425099138
```

b) Using Pandas

```
import pandas as pd
df = pd.read_excel('1.STAT.xlsx')

a = df ['Marks'].mean()
b = df ['Marks'].median()
c = df ['Marks'].mode()
d = df ['Marks'].std()
e = df ['Marks'].max()
f = df ['Marks'].min()
print("Mean=",a)
print("Median=",b)
print("Mode=",c)
print("STD=",d)
print("Max=",e)
print("Min=",f)
```

```
Mean= 63.05
Median= 67.5
Mode= 0    45
Name: Marks, dtype: int64
STD= 24.76303484504357
Max= 100
Min= 10
```

c) Using Numpy

```
import numpy as np
val = [30,40,70,40,50,60,80]
a = np.mean(val)
b = np.median(val)
c = np.std(val)
d = np.max(val)
e = np.min(val)
print("Mean=",a)
print("Median=",b)
print("STD=",c)
print("Max=",d)
print("Min=",e)
```

```
Mean= 52.857142857142854
Median= 50.0
STD= 16.65986255670086
Max= 80
Min= 30
```

PRG 2: Write a program to Load and explore the dataset of .CVS and excel files using pandas and performing the different operations.

A) BASIC OPERATIONS:

a) create a sample Data frame.

b) Writing to CSV file

c) Reading the CSV file

d) Writing to Excel file

e) Reading the Excel file

B) FILTERING DATA:

a) Extracting columns from data set

b) Extracting rows from data set

A) BASIC OPERATIONS:

a) create a sample Data frame.

```
[1]: import pandas as pd

[12]: data = pd.DataFrame({'id':[11,12,13,14,15,16,17,18,19,20],'age':[15,20,25,26,37,31,49,0,0,28],'gender':['M','F','F','M','M','M','F','M','M','F'],'occupa

[13]: data
```

[13]:

	id	age	gender	occupation
0	11	15	M	Manager
1	12	20	F	Teacher
2	13	25	F	Mechanic
3	14	26	M	Lawyer
4	15	37	M	Nurse
5	16	31	M	Doctor
6	17	49	F	Police
7	18	0	M	Hr
8	19	0	M	Salesman
9	20	28	F	Accountant

b) Writing to CSV file
c) Reading the CSV file

[14]:
```
#Writing to CSV File
data.to_csv('data.csv')
```

[25]:
```
#Reading the CSV File
df = pd.read_csv('data.csv')
df
```

[25]:

	Unnamed: 0	id	age	gender	occupation
0	0	11	15	M	Manager
1	1	12	20	F	Teacher
2	2	13	25	F	Mechanic
3	3	14	26	M	Lawyer
4	4	15	37	M	Nurse
5	5	16	31	M	Doctor
6	6	17	49	F	Police
7	7	18	0	M	Hr
8	8	19	0	M	Salesman
9	9	20	28	F	Accountant

d) Writing to Excel file
e) Reading the Excel file

[26]:
```
#Writing to Excel File
data.to_excel('data2.xlsx')
```

[27]:
```
#Reading the Excel File
df2 = pd.read_excel('data2.xlsx')
df2
```

[27]:

	Unnamed: 0	id	age	gender	occupation
0	0	11	15	M	Manager
1	1	12	20	F	Teacher
2	2	13	25	F	Mechanic
3	3	14	26	M	Lawyer
4	4	15	37	M	Nurse
5	5	16	31	M	Doctor
6	6	17	49	F	Police
7	7	18	0	M	Hr
8	8	19	0	M	Salesman
9	9	20	28	F	Accountant

B) FILTERING DATA:

a) Extracting columns from data set

```
[28]: #Extracting particular columns from the data set
      df = pd.read_csv('data.csv',usecols=['id'])
      df
```

[28]:

	id
0	11
1	12
2	13
3	14
4	15
5	16
6	17
7	18
8	19
9	20

b) Extracting rows from dataset

```
In [11]: # Extracting particular rows from data set
         df = pd.read_csv('data.csv', nrows=4)
         df
```

Out[11]:

	Unnamed: 0	id	age	gender	occupation
0	0	11	15	M	Salesman
1	1	12	20	F	Doctor
2	2	13	25	F	Manager
3	3	14	26	M	Teacher

PRG 3: Write a program to read data (CSV and EXCEL files) using tkinter method and upload method using pandas.

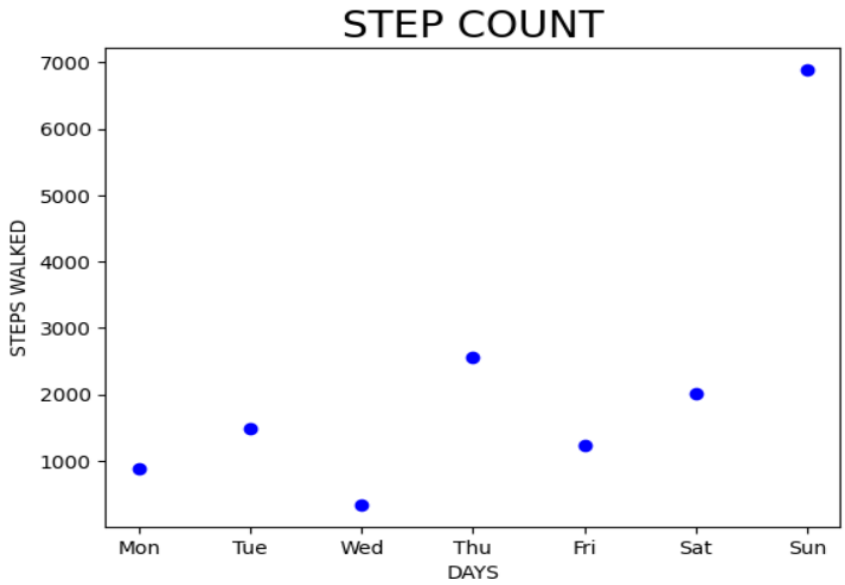
In [*]:
```
import tkinter as tk
from tkinter import filedialog
import pandas as pd

def upload():
    file = filedialog.askopenfilename()

    if file.endswith(".csv"):
        data = pd.read_csv(file)
        print(data)

    elif file.endswith(".xlsx"):
        data = pd.read_excel(file)
        print(data)

root = tk.Tk()

root.title("File Upload")
root.geometry("250x150")

btn = tk.Button(root, text="Upload File", command=upload)
btn.pack(pady=40)

root.mainloop()
```

```
   Age  Glucose
0   43       99
1   21       65
2   25       79
3   42       75
4   57       87
5   59       81
```

PRG 4: Write a program to illustrate Data Visualization using Scatter plot, Bar chart, Line graph, and Pie chart

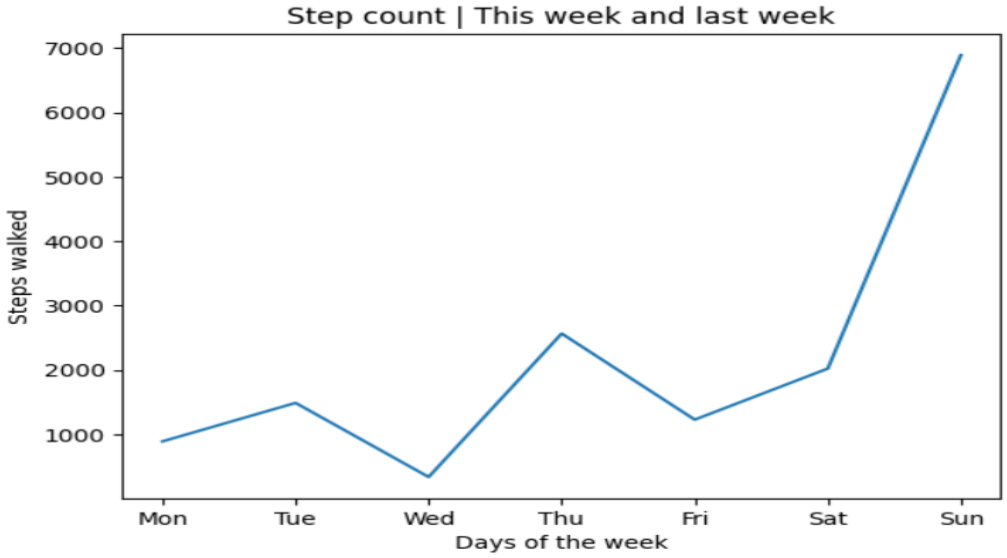
a) Scatter plot

```
#SCATTER PLOT
import matplotlib.pyplot as plt
days = ["Mon","Tue","Wed","Thu","Fri","Sat","Sun"]
steps_walked = [893,1490,340,2567,1230,2023,6890]
plt.scatter(days, steps_walked, c="blue")
plt.title('STEP COUNT', fontsize=20)
plt.ylabel('STEPS WALKED')
plt.xlabel('DAYS')
plt.show()
```



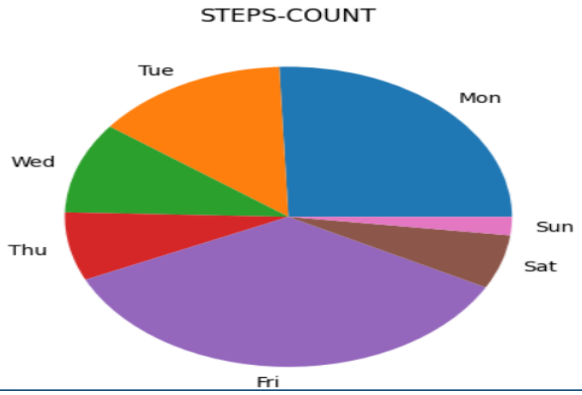
b) Line plot

```
#LINE PLOT
plt.plot(days,steps_walked) #Plot the chart
plt.title("Step count | This week and last week")
plt.xlabel("Days of the week")
plt.ylabel("Steps walked")
plt.show()
```



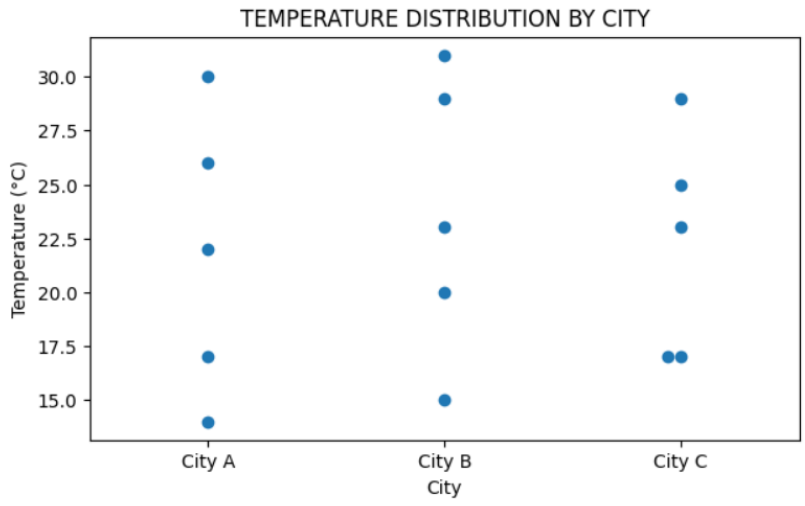
c) Pie chart

```
[5]: #PIE CHART
import pandas as pd
data ={  'days':["Mon","Tue","Wed","Thu","Fri","Sat","Sun"],
       'steps_walked':[8934,4902,3409,2562,12300,2023,689]}

df=pd.DataFrame(data)

#Plot the pie chart

plt.pie(df['steps_walked'],labels=df['days'])
plt.title('STEPS-COUNT')

plt.show()
```



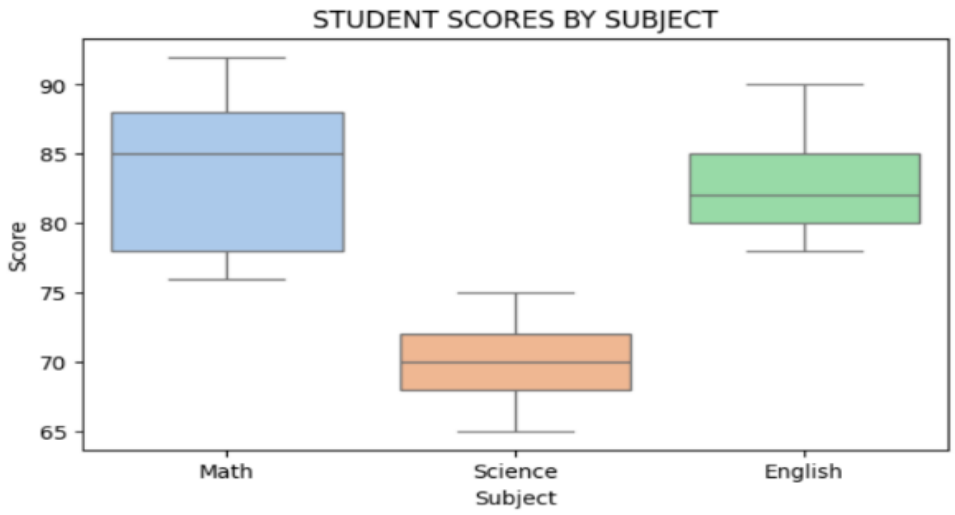
d) Swarm plot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Temperature data by city
data = {
    'city': [
        'City A','City A','City A','City A','City A',
        'City B','City B','City B','City B','City B',
        'City C','City C','City C','City C','City C'
    ],
    'temperature': [
        17, 30, 22, 14, 26,    # City A
        15, 20, 29, 31, 23,    # City B
        25, 17, 29, 17, 23     # City C
    ]
}

df = pd.DataFrame(data)

# SWARM PLOT
plt.figure(figsize=(7,4))
sns.swarmplot(x='city', y='temperature', data=df, size=7)
plt.title('TEMPERATURE DISTRIBUTION BY CITY')
plt.xlabel('City')
plt.ylabel('Temperature (°C)')
plt.show()
```



e) Box plot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset: Exam scores
data = {
    'subject': [
        'Math','Math','Math','Math','Math',
        'Science','Science','Science','Science','Science',
        'English','English','English','English','English'
    ],
    'score': [
        78, 85, 92, 88, 76,     # Math
        65, 70, 72, 68, 75,     # Science
        80, 82, 85, 78, 90      # English
    ]
}

df = pd.DataFrame(data)

# BOX PLOT
plt.figure(figsize=(7,4))
sns.boxplot(x='subject', y='score', data=df, palette='pastel')
plt.title('STUDENT SCORES BY SUBJECT')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```



PRG 5: Write a program to construct CONFUSION MATRIX for the given problem and evaluate the performance of a classification model.

```python
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, precision_score, recall_score, f1_score

# Actual Labels
# 1 = Positive, 0 = Negative
# TP + FN = actual positives = 8 + 3 = 11
# TN + FP = actual negatives = 5 + 4 = 9
y_true = [1]*11 + [0]*9

# Predicted Labels
# TP = 8, FN = 3, FP = 4, TN = 5
y_pred = [1]*8 + [0]*3 + [1]*4 + [0]*5

# Confusion Matrix
cm = confusion_matrix(y_true, y_pred)
print("Confusion Matrix:")
print(cm)

# Evaluation Metrics
accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred)
recall = recall_score(y_true, y_pred)
f1 = f1_score(y_true, y_pred)

print("\nEvaluation Metrics:")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall :", recall)
print("F1-Score :", f1)

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_true, y_pred))
```

```
Confusion Matrix:
[[5 4]
 [3 8]]

Evaluation Metrics:
Accuracy : 0.65
Precision: 0.6666666666666666
Recall : 0.7272727272727273
F1-Score : 0.6956521739130435

Classification Report:
              precision    recall  f1-score   support

           0       0.62      0.56      0.59         9
           1       0.67      0.73      0.70        11

    accuracy                           0.65        20
   macro avg       0.65      0.64      0.64        20
weighted avg       0.65      0.65      0.65        20
```

PRG 6: Write a program to illustrate DATA PREPROCESSING [Student Data]

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("STUD.csv")
```

[3]:
```
print("Original Dataset:\n")
print(data)
```

```
Original Dataset:

        NAME   AGE  GENDER  MARKS       CITY
0      VIRAT  18.0    MALE   99.0  BENGALURU
1      ROHIT  19.0    MALE   98.0  MANGALURU
2     DHAWAN  20.0    MALE   96.0        NaN
3     WARNER   NaN    MALE   45.0  MANGALURU
4      GAYLE  14.0    MALE   12.0  BENGALURU
5        ABD  12.0    MALE   23.0        NaN
6    MAXWELL  21.0    MALE    NaN     MYSORE
7       LARA   NaN  FEMALE   34.0        NaN
8    KULDEEP  21.0    MALE    NaN     MYSORE
9     BUMRAH   NaN    MALE   45.0        NaN
10  ARSHDEEP  20.0    MALE   56.0  BENGALURU
11    MILLER   NaN  FEMALE    NaN        NaN
12      SALT  16.0  FEMALE   67.0     MYSORE
13    KRUNAL  15.0    MALE   65.0  MANGALURU
14    HARDIK  13.0    MALE   34.0  BENGALURU
15     SURYA  13.0  FEMALE    NaN        NaN
16    HARMAN   NaN  FEMALE   23.0  BENGALURU
17     SANJU  20.0    MALE   45.0        NaN
18    RASHID  20.0    MALE    NaN    CHENNAI
19    SMRITI  19.0  FEMALE   56.0    CHENNAI
20     VARUN   NaN    MALE   78.0    CHENNAI
21     BROCK   NaN  FEMALE   89.0  BENGALURU
22   GUNTHER  11.0  FEMALE    NaN  BENGALURU
23       JEY  10.0    MALE    NaN        NaN
24     JIMMY  10.0    MALE   67.0  MANGALURU
```

[5]:
```
df = pd.DataFrame(data)
df
```

[5]:

	NAME	AGE	GENDER	MARKS	CITY
0	VIRAT	18.0	MALE	99.0	BENGALURU
1	ROHIT	19.0	MALE	98.0	MANGALURU
2	DHAWAN	20.0	MALE	96.0	NaN
3	WARNER	NaN	MALE	45.0	MANGALURU
4	GAYLE	14.0	MALE	12.0	BENGALURU
5	ABD	12.0	MALE	23.0	NaN
6	MAXWELL	21.0	MALE	NaN	MYSORE
7	LARA	NaN	FEMALE	34.0	NaN
8	KULDEEP	21.0	MALE	NaN	MYSORE
9	BUMRAH	NaN	MALE	45.0	NaN
10	ARSHDEEP	20.0	MALE	56.0	BENGALURU
11	MILLER	NaN	FEMALE	NaN	NaN
12	SALT	16.0	FEMALE	67.0	MYSORE
13	KRUNAL	15.0	MALE	65.0	MANGALURU
14	HARDIK	13.0	MALE	34.0	BENGALURU
15	SURYA	13.0	FEMALE	NaN	NaN
16	HARMAN	NaN	FEMALE	23.0	BENGALURU
17	SANJU	20.0	MALE	45.0	NaN

[6]:
```
data.isnull().sum()
```

[6]:
```
NAME      0
AGE       7
GENDER    0
MARKS     7
CITY      8
dtype: int64
```

[7]:
```
df2 = df.copy()
```

[8]:
```
#Shows only the row/column has a missing value
df2[df2.isnull().any(axis=1)]
```

[8]:

	NAME	AGE	GENDER	MARKS	CITY
2	DHAWAN	20.0	MALE	96.0	NaN
3	WARNER	NaN	MALE	45.0	MANGALURU
5	ABD	12.0	MALE	23.0	NaN
6	MAXWELL	21.0	MALE	NaN	MYSORE
7	LARA	NaN	FEMALE	34.0	NaN
8	KULDEEP	21.0	MALE	NaN	MYSORE
9	BUMRAH	NaN	MALE	45.0	NaN
11	MILLER	NaN	FEMALE	NaN	NaN
15	SURYA	13.0	FEMALE	NaN	NaN
16	HARMAN	NaN	FEMALE	23.0	BENGALURU
17	SANJU	20.0	MALE	45.0	NaN
18	RASHID	20.0	MALE	NaN	CHENNAI
20	VARUN	NaN	MALE	78.0	CHENNAI
21	BROCK	NaN	FEMALE	89.0	BENGALURU
22	GUNTHER	11.0	FEMALE	NaN	BENGALURU
23	JEY	10.0	MALE	NaN	NaN

[9]:
```
#Handling the missing values

#Remove rows with any missing value
df_row_deleted = df.dropna()

print("\nAfter Row Deletion:\n")
print(df_row_deleted)
```

```
After Row Deletion:

        NAME   AGE  GENDER  MARKS       CITY
0      VIRAT  18.0    MALE   99.0  BENGALURU
1      ROHIT  19.0    MALE   98.0  MANGALURU
4      GAYLE  14.0    MALE   12.0  BENGALURU
10  ARSHDEEP  20.0    MALE   56.0  BENGALURU
12      SALT  16.0  FEMALE   67.0     MYSORE
13    KRUNAL  15.0    MALE   65.0  MANGALURU
14    HARDIK  13.0    MALE   34.0  BENGALURU
19    SMRITI  19.0  FEMALE   56.0    CHENNAI
24     JIMMY  10.0    MALE   67.0  MANGALURU
```

```
[10]: df_row_deleted.isnull().sum()
```

```
[10]: NAME      0
      AGE       0
      GENDER    0
      MARKS     0
      CITY      0
      dtype: int64
```

```
[11]: df_column_deleted = df.dropna(axis=1)

      print("\nAfter Column Deletion:\n")
      print(df_column_deleted)
```

```
After Column Deletion:

         NAME  GENDER
0       VIRAT    MALE
1       ROHIT    MALE
2      DHAWAN    MALE
3      WARNER    MALE
4       GAYLE    MALE
5         ABD    MALE
6     MAXWELL    MALE
7        LARA  FEMALE
8     KULDEEP    MALE
9      BUMRAH    MALE
10   ARSHDEEP    MALE
11     MILLER  FEMALE
12       SALT  FEMALE
13     KRUNAL    MALE
14     HARDIK    MALE
15      SURYA  FEMALE
16     HARMAN  FEMALE
17      SANJU    MALE
18     RASHID    MALE
19     SMRITI  FEMALE
20      VARUN    MALE
21      BROCK  FEMALE
22    GUNTHER  FEMALE
23        JEY    MALE
24      JIMMY    MALE
```

```
[12]: df_column_deleted.isnull().sum()
```

```
[12]: NAME      0
      GENDER    0
      dtype: int64
```

[13]:
```
df_mean = df.copy()

df_mean['AGE'].fillna(df_mean['AGE'].mean(),inplace=True)
df_mean['MARKS'].fillna(df_mean['MARKS'].mean(),inplace=True)

print("\nAfter Mean Imputation\n")
print(df_mean)
```

```
After Mean Imputation

        NAME   AGE  GENDER  MARKS       CITY
0      VIRAT  18.0    MALE   99.0  BENGALURU
1      ROHIT  19.0    MALE   98.0  MANGALURU
2     DHAWAN  20.0    MALE   96.0        NaN
3     WARNER   NaN    MALE   45.0  MANGALURU
4      GAYLE  14.0    MALE   12.0  BENGALURU
5        ABD  12.0    MALE   23.0        NaN
6    MAXWELL  21.0    MALE    NaN     MYSORE
7       LARA   NaN  FEMALE   34.0        NaN
8    KULDEEP  21.0    MALE    NaN     MYSORE
9     BUMRAH   NaN    MALE   45.0        NaN
10  ARSHDEEP  20.0    MALE   56.0  BENGALURU
11    MILLER   NaN  FEMALE    NaN        NaN
12      SALT  16.0  FEMALE   67.0     MYSORE
13    KRUNAL  15.0    MALE   65.0  MANGALURU
14    HARDIK  13.0    MALE   34.0  BENGALURU
15     SURYA  13.0  FEMALE    NaN        NaN
16    HARMAN   NaN  FEMALE   23.0  BENGALURU
17     SANJU  20.0    MALE   45.0        NaN
18    RASHID  20.0    MALE    NaN    CHENNAI
19    SMRITI  19.0  FEMALE   56.0    CHENNAI
20     VARUN   NaN    MALE   78.0    CHENNAI
21     BROCK   NaN  FEMALE   89.0  BENGALURU
22   GUNTHER  11.0  FEMALE    NaN  BENGALURU
23       JEY  10.0    MALE    NaN        NaN
```

[14]:
```
df_median = df.copy()

df_median['AGE'].fillna(df_median['AGE'].median(),inplace=True)
df_median['MARKS'].fillna(df_median['MARKS'].median(),inplace=True)

print("\nAfter Median Imputation\n")
print(df_median)
```

```
After Median Imputation

        NAME   AGE  GENDER  MARKS       CITY
0      VIRAT  18.0    MALE   99.0  BENGALURU
1      ROHIT  19.0    MALE   98.0  MANGALURU
2     DHAWAN  20.0    MALE   96.0        NaN
3     WARNER   NaN    MALE   45.0  MANGALURU
4      GAYLE  14.0    MALE   12.0  BENGALURU
5        ABD  12.0    MALE   23.0        NaN
6    MAXWELL  21.0    MALE    NaN     MYSORE
7       LARA   NaN  FEMALE   34.0        NaN
8    KULDEEP  21.0    MALE    NaN     MYSORE
9     BUMRAH   NaN    MALE   45.0        NaN
10  ARSHDEEP  20.0    MALE   56.0  BENGALURU
11    MILLER   NaN  FEMALE    NaN        NaN
12      SALT  16.0  FEMALE   67.0     MYSORE
13    KRUNAL  15.0    MALE   65.0  MANGALURU
14    HARDIK  13.0    MALE   34.0  BENGALURU
15     SURYA  13.0  FEMALE    NaN        NaN
16    HARMAN   NaN  FEMALE   23.0  BENGALURU
17     SANJU  20.0    MALE   45.0        NaN
18    RASHID  20.0    MALE    NaN    CHENNAI
19    SMRITI  19.0  FEMALE   56.0    CHENNAI
20     VARUN   NaN    MALE   78.0    CHENNAI
21     BROCK   NaN  FEMALE   89.0  BENGALURU
22   GUNTHER  11.0  FEMALE    NaN  BENGALURU
23       JEY  10.0    MALE    NaN        NaN
```

[15]:
```
df_mode = df.copy()

df_mode['CITY'].fillna(df_mode['CITY'].mode()[0],inplace=True)
df_mode['GENDER'].fillna(df_mode['GENDER'].mode()[0],inplace=True)
df_mode['AGE'].fillna(df_mode['AGE'].mode()[0],inplace=True)
df_mode['MARKS'].fillna(df_mode['MARKS'].mode()[0],inplace=True)

print("After Mode Imputation:\n")
print(df_mode)
```

```
After Mode Imputation:

        NAME   AGE  GENDER  MARKS       CITY
0      VIRAT  18.0    MALE   99.0  BENGALURU
1      ROHIT  19.0    MALE   98.0  MANGALURU
2     DHAWAN  20.0    MALE   96.0        NaN
3     WARNER   NaN    MALE   45.0  MANGALURU
4      GAYLE  14.0    MALE   12.0  BENGALURU
5        ABD  12.0    MALE   23.0        NaN
6    MAXWELL  21.0    MALE    NaN     MYSORE
7       LARA   NaN  FEMALE   34.0        NaN
8    KULDEEP  21.0    MALE    NaN     MYSORE
9     BUMRAH   NaN    MALE   45.0        NaN
10  ARSHDEEP  20.0    MALE   56.0  BENGALURU
11    MILLER   NaN  FEMALE    NaN        NaN
12      SALT  16.0  FEMALE   67.0     MYSORE
13    KRUNAL  15.0    MALE   65.0  MANGALURU
14    HARDIK  13.0    MALE   34.0  BENGALURU
15     SURYA  13.0  FEMALE    NaN        NaN
16    HARMAN   NaN  FEMALE   23.0  BENGALURU
17     SANJU  20.0    MALE   45.0        NaN
18    RASHID  20.0    MALE    NaN    CHENNAI
19    SMRITI  19.0  FEMALE   56.0    CHENNAI
20     VARUN   NaN    MALE   78.0    CHENNAI
21     BROCK   NaN  FEMALE   89.0  BENGALURU
22   GUNTHER  11.0  FEMALE    NaN  BENGALURU
```

[16]:
```
#Encoding Categorical Data(Gender & City)

#Label Encoding
le = LabelEncoder()
df_mode['GENDER'] = le.fit_transform(df_mode['GENDER'].astype(str))
df_mode['CITY'] = le.fit_transform(df_mode['CITY'].astype(str))

print("\nAfter Encoding Categorical Data:\n ")
print(df_mode)
```

```
After Encoding Categorical Data:

        NAME   AGE  GENDER  MARKS  CITY
0      VIRAT  18.0       1   99.0     0
1      ROHIT  19.0       1   98.0     2
2     DHAWAN  20.0       1   96.0     4
3     WARNER   NaN       1   45.0     2
4      GAYLE  14.0       1   12.0     0
5        ABD  12.0       1   23.0     4
6    MAXWELL  21.0       1    NaN     3
7       LARA   NaN       0   34.0     4
8    KULDEEP  21.0       1    NaN     3
9     BUMRAH   NaN       1   45.0     4
10  ARSHDEEP  20.0       1   56.0     0
11    MILLER   NaN       0    NaN     4
12      SALT  16.0       0   67.0     3
13    KRUNAL  15.0       1   65.0     2
14    HARDIK  13.0       1   34.0     0
15     SURYA  13.0       0    NaN     4
16    HARMAN   NaN       0   23.0     0
17     SANJU  20.0       1   45.0     4
18    RASHID  20.0       1    NaN     1
19    SMRITI  19.0       0   56.0     1
20     VARUN   NaN       1   78.0     1
21     BROCK   NaN       0   89.0     0
```

[17]:
```
#Feature Scaling (Normalization)
scaler = MinMaxScaler()
```

[18]:
```
#Scaling Age and Marks columns
df_mode[['AGE','MARKS']] = scaler.fit_transform(df_mode[['AGE','MARKS']])
```

[19]:
```
print("After Normalization\n")
print(df_mode)
```

```
After Normalization

        NAME       AGE  GENDER     MARKS  CITY
0      VIRAT  0.727273       1  1.000000     0
1      ROHIT  0.818182       1  0.988506     2
2     DHAWAN  0.909091       1  0.965517     4
3     WARNER       NaN       1  0.379310     2
4      GAYLE  0.363636       1  0.000000     0
5        ABD  0.181818       1  0.126437     4
6    MAXWELL  1.000000       1       NaN     3
7       LARA       NaN       0  0.252874     4
8    KULDEEP  1.000000       1       NaN     3
9     BUMRAH       NaN       1  0.379310     4
10  ARSHDEEP  0.909091       1  0.505747     0
11    MILLER       NaN       0       NaN     4
12      SALT  0.545455       0  0.632184     3
13    KRUNAL  0.454545       1  0.609195     2
14    HARDIK  0.272727       1  0.252874     0
15     SURYA  0.272727       0       NaN     4
16    HARMAN       NaN       0  0.126437     0
17     SANJU  0.909091       1  0.379310     4
18    RASHID  0.909091       1       NaN     1
19    SMRITI  0.818182       0  0.505747     1
20     VARUN       NaN       1  0.758621     1
21     BROCK       NaN       0  0.885057     0
22   GUNTHER  0.090909       0       NaN     0
```

[20]:
```
#Suppose we predict Marks
X = df_mode[['AGE','GENDER','CITY']]
Y = df_mode['MARKS']
```

[21]:
```
#Train-Test Split
X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size=0.2, random_state=0)

print("\nTraining Data:\n",X_train)
print("\nTesting Data:\n",X_test)
```

```
Training Data:
         AGE  GENDER  CITY
22  0.090909       0     0
17  0.909091       1     4
24  0.000000       1     2
23  0.000000       1     4
14  0.272727       1     0
1   0.818182       1     2
10  0.909091       1     0
13  0.454545       1     2
8   1.000000       1     3
6   1.000000       1     3
18  0.909091       1     1
4   0.363636       1     0
9        NaN       1     4
7        NaN       0     4
20       NaN       1     1
3        NaN       1     2
0   0.727273       1     0
21       NaN       0     0
15  0.272727       0     4
12  0.545455       0     3

Testing Data:
         AGE  GENDER  CITY
5   0.181818       1     4
2   0.909091       1     4
19  0.818182       0     1
16       NaN       0     0
11       NaN       0     4
```

PRG 7: Demonstrate KNN classification for the given dataset

[1]:
```
# KNN Student Result Prediction using Excel Data

# Step 1: Import Libraries
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
```

[3]:
```
# Step 2: Read Excel file
data = pd.read_excel("1.STUDENT_DATA.xlsx")
```

[4]:
```
# Step 3: Display dataset
print("\nDataset:\n")
print(data)
```

```
Dataset:

   Unnamed: 0  StudyHours  Attendance Result
0           0           2          40   Fail
1           1           4          45   Fail
2           2           5          60   Pass
3           3           6          65   Pass
4           4           7          70   Pass
5           5           3          50   Fail
```

[5]:
```
# Step 4: Separate features and Label
X = data[['StudyHours', 'Attendance']] # Input features
y = data['Result'] # Target output
```

[6]:
```
# Step 5: Create KNN Model (K=3, Euclidean distance)
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
```

[12]:
```
# Step 6: Train the model
knn.fit(X, y)
```

[12]:

KNeighborsClassifier

▼ Parameters

n_neighbors	3
weights	'uniform'
algorithm	'auto'
leaf_size	30
p	2
metric	'euclidean'
metric_params	None
n_jobs	None

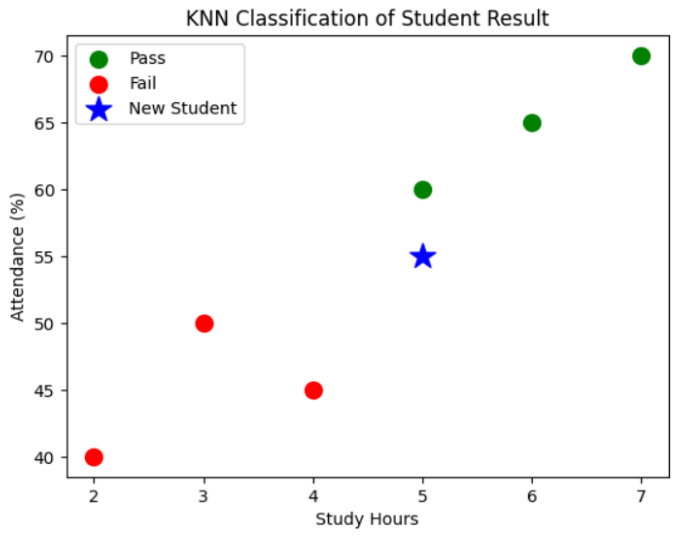
[13]:
```
# Step 7: New student data (Test data)
# StudyHours = 5, Attendance = 55
new_student = [[5, 55]]
```

[16]:
```
# Step 8: Predict result
prediction = knn.predict(new_student)
import warnings
warnings.filterwarnings("ignore")
```

[17]:
```
# Step 9: Display result
print(f"\nNew Student Data (StudyHours 5, Attendance 55)")
print("Predicted Result:", prediction[0])
```

```
New Student Data (StudyHours 5, Attendance 55)
Predicted Result: Pass
```

[18]:
```
import matplotlib.pyplot as plt

# --- Visualization Section ---

# Separate Pass and Fail students for plotting
pass_students = data[data['Result'] == 'Pass']
fail_students = data[data['Result'] == 'Fail']

# Plot Pass students (Green)
plt.scatter(pass_students['StudyHours'],
            pass_students['Attendance'],
            color='green',
            label='Pass',
            s=100)

# Plot Fail students (Red)
plt.scatter(fail_students['StudyHours'],
            fail_students['Attendance'],
            color='red',
            label='Fail',
            s=100)

# Plot New Student (Blue Star)
plt.scatter(5, 55,
            color='blue',
            marker='*',
            s=250,
            label='New Student')

# Labels and Title
plt.xlabel("Study Hours")
plt.ylabel("Attendance (%)")
plt.title("KNN Classification of Student Result")
# Legend and Show graph
plt.legend()
plt.show()
```



PRG 8: Write a program to demonstrate simple linear regression for the given dataset.

[1]:
```
# Step 1: Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

[2]:
```
# Step 2: Load File
file = "sum1.csv"
data = pd.read_csv(file)
```

[3]:
```
#Preprocessing
data.dropna(inplace=True)

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

#Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

[4]:
```
#Step 3: Data Preprocessing
print("Dataset:\n", data)
```

```
Dataset:
    Age  Glucose
0   43       99
1   21       65
2   25       79
3   42       75
4   57       87
5   59       81
```

[5]:
```
print(X_train.dtypes)
print(y_train.dtypes)
```

```
Age    int64
dtype: object
int64
```

[6]:
```
#Remove extra spaces in column names
data.columns = data.columns.str.strip()
```

[7]:
```
 #Check missing values
print("\nMissing values:\n", data.isnull().sum())
```

```
Missing values:
 Age        0
Glucose    0
dtype: int64
```

[8]:
```
#Drop missing values
data = data.dropna()
```

[9]:
```
#Convert to numeric (safety step)
data['Age'] = pd.to_numeric(data['Age'], errors='coerce')
data['Glucose'] = pd.to_numeric(data['Glucose'], errors='coerce')
```

[10]:
```
#Step 4: Define Features and Target
X = data[['Age']] #Indepenedent variable
y = data['Glucose'] #Depenedent variable
```

[11]:
```
#Step 5: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

[12]:
```
#Step 6: Create and Train Model
model = LinearRegression()
model.fit(X_train, y_train)
```

[12]:

LinearRegression

▾ Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

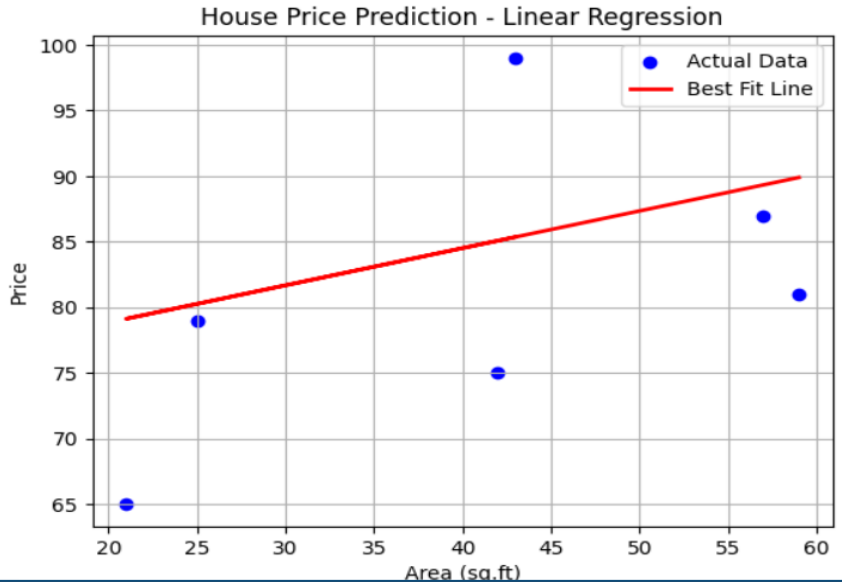
[13]:
```
#Step 7: Get Slope and Intercept
slope = model.coef_[0]
intercept = model.intercept_

print("\nSlope (b):", slope)
print("Intercept (a):", intercept)

#Regression Equation
print(f"\nRegression Equation: Y = {intercept:.2f} + {slope:.2f}X")
```

```
Slope (b): 0.2836328314715882
Intercept (a): 73.15832928606119

Regression Equation: Y = 73.16 + 0.28X
```

```
#Step 8: Prediction
area = float(input("\nEnter Area to predict price:"))
predicted_price = model.predict([[area]])

print(f"Predicted Price for {area} sq.ft = {predicted_price[0]:.2f}")
```

```
Enter Area to predict price: 55
Predicted Price for 55.0 sq.ft = 88.76
```

[15]:
```
#Step 9: Best Fit Line Graph
plt.scatter(X,y,color='blue',label="Actual Data")
plt.plot(X, model.predict(X), color='red', linewidth=2, label="Best Fit Line")

plt.xlabel("Area (sq.ft)")
plt.ylabel("Price")
plt.title("House Price Prediction - Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```



PRG 9: Write a program to Implement K-Means clustering and Visualize clusters for the given dataset.

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

[2]:
```
#1. Read CSV File
data = pd.read_csv("kmean1.csv")
```

[3]:
```
#2. Add Labels like A1, A2, A3, ...
data ['Label'] = ['a'+str(i+1) for i in range(len(data))]
```

[4]:
```
X = data[['X', 'Y']]
```

[5]:
```
#4. Apply K-Means
kmeans = KMeans(n_clusters = 2, random_state = 0)
kmeans.fit(X)
```

[5]:
```
▾ KMeans
▸ Parameters
```

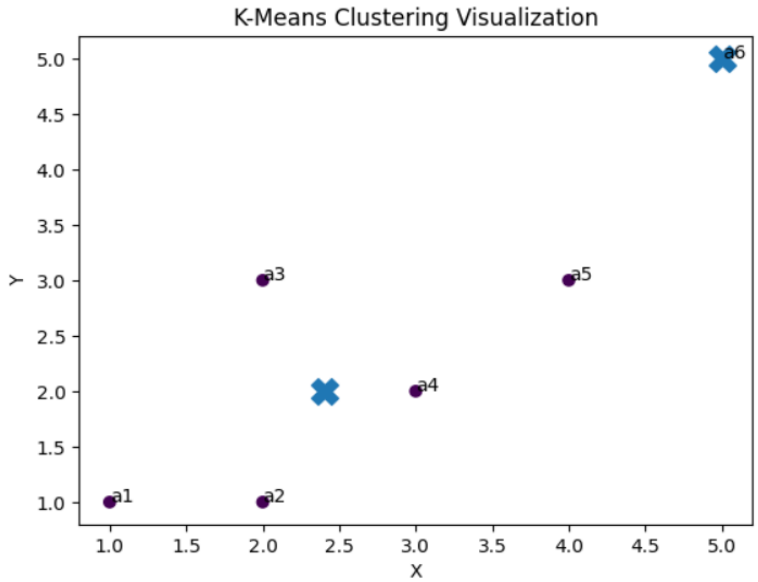
[6]:
```
data['Cluster'] = kmeans.labels_
centroids = kmeans.cluster_centers_
```

[7]:
```
#6. Print final clusters
clusters = data.groupby('Cluster')['Label'].apply(list)
```

[8]:
```
for i in clusters.index:
    print(f"Cluster {i+1}:{', '.join(clusters[i])}")
```

```
Cluster 1:a1, a2, a3, a4, a5
Cluster 2:a6
```

```
plt.scatter(X['X'], X['Y'], c=data['Cluster'])
plt.scatter(centroids[:, 0], centroids[:, 1], marker="X", s=200)
#Annotate points with labels (a1, a2, ...)
for i in range(len(data)):
    plt.text(X.iloc[i,0], X.iloc[i,1], data['Label'][i])
plt.title("K-Means Clustering Visualization")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



PRG 10: Write a program to demonstrate Naïve Bayes Classification algorithm for the given

[2]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
```

[3]:
```
data = pd.read_csv("Weather.csv")
data
```

[3]:

	Outlook	Play
0	Rainy	Yes
1	Sunny	Yes
2	Overcast	Yes
3	Overcast	Yes
4	Sunny	No
5	Rainy	Yes
6	Sunny	Yes
7	Overcast	Yes
8	Rainy	No
9	Sunny	No
10	Sunny	Yes
11	Rainy	No
12	Overcast	Yes
13	Overcast	Yes

[4]:
```
x =data[['Outlook']]
y=data ['Play']
```

[5]:
```
le_x = LabelEncoder()
le_y = LabelEncoder()
```

[6]:
```
x['Outlook'] = le_x.fit_transform(x['Outlook'])
y = le_y.fit_transform(y)
```

[7]:
```
model = GaussianNB()
model.fit(x,y)
```

[7]:

GaussianNB

▼ Parameters

priors	None
var_smoothing	1e-09

[8]:
```
new_data = pd.DataFrame({'Outlook':['Sunny']})
new_data['Outlook'] = le_x.transform(new_data['Outlook'])
```

[9]:
```
prediction = model.predict(new_data)
result = le_y.inverse_transform(prediction)
```

[10]:
```
print("Prediction:",result[0])
```

```
Prediction: No
```